

Assignment 3: Architecture Diagram

Trace — Personal Productivity and Task Execution Agent

Prepared by: Maryam Fatima

System Architecture Diagram

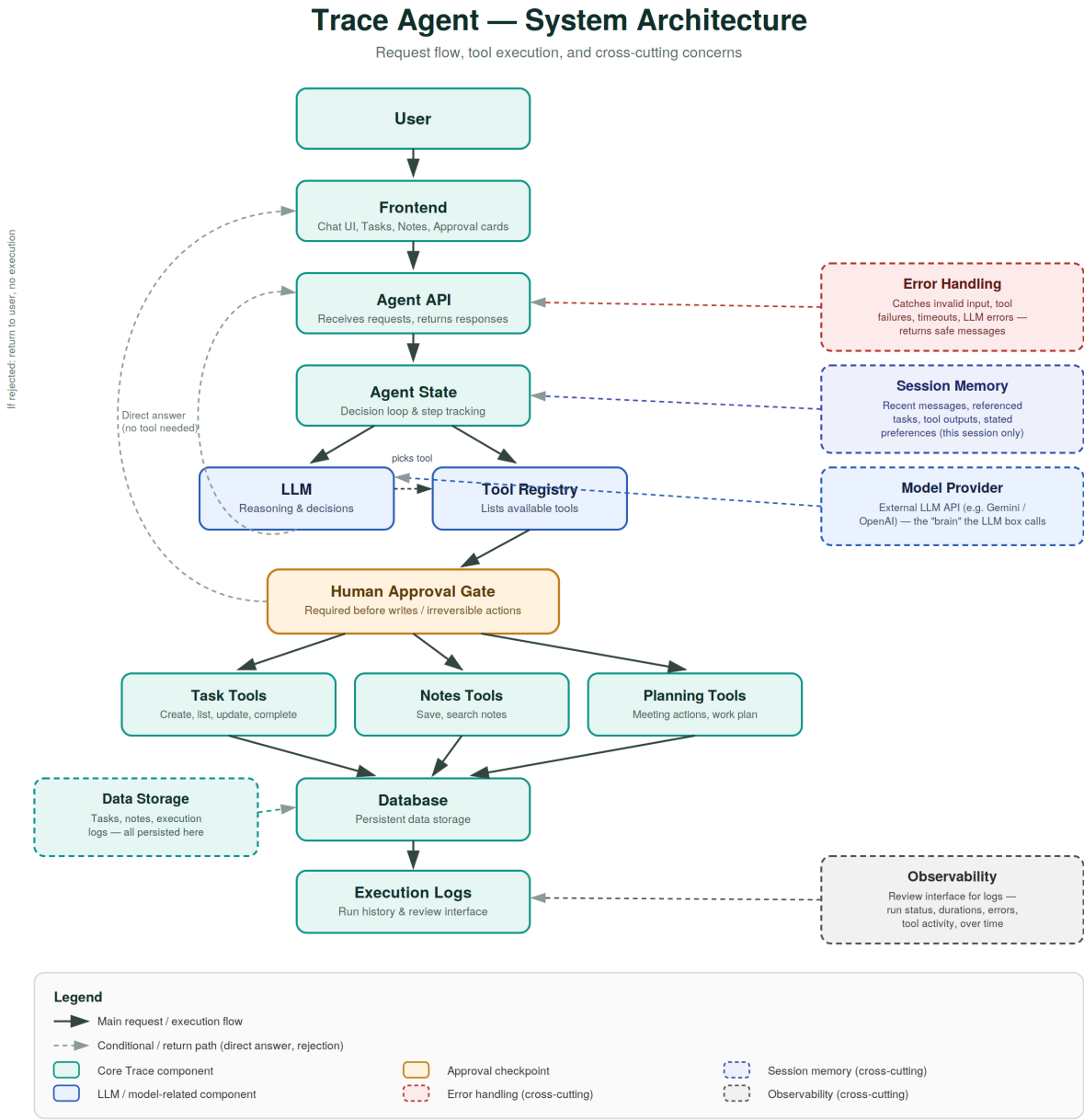


Figure 1: End-to-end flow from the user's request through the frontend, the agent's decision loop, tool execution, human approval, and persistent storage — with error handling, session memory, the model provider, and observability shown as supporting components.

Component Explanations

This section explains, in simple terms, what each part of the diagram does and how it connects to the rest of the system.

User

The person using Trace. They type a request in plain language, such as asking for a work plan, saving a note, or updating a task. The user never talks to the backend directly — every request goes through the frontend.

Frontend

The interface the user sees and interacts with: the chat window, the Tasks board, the Notes view, and the approval cards. It sends the user's message to the Agent API and displays whatever comes back, including approval requests when the agent wants to perform a write action.

Agent API

The entry point on the backend. It receives each request from the frontend, passes it into the agent's decision loop, and sends the final response back. It is also the first place where basic input problems (like an empty message) are caught.

Agent State

The part of the system that runs the decision loop for a single request. It keeps track of how many steps have been taken so far, what has already been tried, and whether the agent should keep going, ask a tool for help, or stop and answer directly. It enforces the execution limits, such as the maximum number of steps and duplicate tool-call detection, so the agent cannot loop forever.

LLM

The language model that does the actual reasoning. Given the user's request and the list of available tools, it decides whether a tool is needed at all, and if so, which one and with what arguments. For a simple question like explaining the difference between priority levels, the LLM answers directly and no tool is called.

Tool Registry

A catalogue of every tool the agent is allowed to use, grouped by what they do. The LLM consults this registry to pick the right tool rather than guessing one from scratch. This is also where invalid tool names or malformed arguments are detected before anything is executed.

Human Approval Gate

A required checkpoint before any write or irreversible action runs — creating or updating a task, completing a task, saving a note, or sending a reminder/email. The user sees the proposed action, the tool name, the exact input arguments, and the expected effect, then chooses Approve, Reject, or Edit. If rejected, control returns to the user immediately and the tool never runs.

Task Tools

The group of tools that manage tasks: creating a task, listing tasks with filters (status, priority, due date, tag), updating a task, and marking a task complete. Update and Complete both require approval since

they change stored data.

Notes Tools

The group of tools that manage notes: saving a new note and searching existing notes by keyword, category, or date range.

Planning Tools

The group of tools that turn raw information into structured plans: extracting decisions and action items from meeting notes, and generating an ordered work plan from the user's available hours, tasks, and priorities.

Database

Where everything is stored persistently — tasks, notes, and execution logs. Data written here survives after the session ends, so nothing is lost when the app is closed and reopened.

Data Storage

The same persistence layer viewed from the data side: it holds every task, note, and logged run in a durable form, not just in memory, which is what makes reloading the app show the same data as before.

Execution Logs

A record kept for every agent run: the run ID, the original request, the model used, which tools were called and with what arguments, the results, the approval status, any errors, start and end time, total duration, and the final outcome. Private reasoning and API keys are never written to this log.

Observability

The interface that lets the user look back through the execution logs — reviewing past runs, seeing how long they took, which tools ran, and where something failed. This is how problems get noticed and understood after the fact, not just in the moment.

Error Handling

A layer that runs alongside every step of the flow, not just one box in the chain. It catches problems such as a missing API key, an invalid model response, bad tool arguments, an unknown task ID, a database failure, a tool timeout, or repeated failed tool calls — and turns each one into a clear, understandable message for the user instead of a raw error or stack trace.

Session Memory

Short-term memory kept only for the current session: the recent messages, which tasks the user has referred to, the results of previous tool calls, and any preferences the user has stated. This is what allows a follow-up like “mark the second one as complete” to correctly resolve which task “the second one” means, based on the list shown a moment earlier.

Model Provider

The external LLM API that the LLM box actually calls to generate its reasoning and decisions (for example Gemini or OpenAI). It is treated as a separate, swappable component so the rest of the system does not depend on any one provider.

How a Request Flows Through the System

The user sends a request through the frontend, which reaches the Agent API and enters the Agent State's decision loop. The LLM, using the Tool Registry, decides whether a tool is needed at all. If the request can be answered directly (for example, a factual question), the LLM answers immediately and the flow returns straight to the frontend — no tool is touched. If a tool is needed, the LLM picks one from the correct category (Task, Notes, or Planning). Before any write or irreversible tool actually runs, the Human Approval Gate shows the user exactly what is about to happen; the user approves, edits, or rejects it. Approved actions run against the Database, which persists the change, and every run — approved or not — is recorded in the Execution Logs, which the user can review through Observability. Error Handling and Session Memory operate throughout this entire path rather than at a single point, and the Model Provider is the external service the LLM relies on to reason.